

Software Implementation and Testing Document

For

Group 3

Version 2.0

Authors:

Tyler Drake
Christian Wilsey
Luis Villanueva
Emmanuel Reyes Martinez
Jamie Velasquez

1. Programming Languages (5 points)

List the programming languages use in your project, where you use them (what components of your project) and your reason for choosing them (whatever that may be).

JavaScript

Used for:

- Express backend server
- API routes
- Skinport API service
- Scheduled refresh job
- PostgreSQL repository functions
- In-memory cache logic

Reason chosen:

JavaScript was chosen because it works well with Node.js and Express for building REST APIs. It also allowed the backend and frontend portions of the project to use a similar language ecosystem.

SQL

Used for:

- PostgreSQL schema
- Creating database tables
- Defining indexes
- Querying item data
- Inserting and updating item records

Reason chosen:

SQL was chosen because the project requires structured storage for item metadata, latest prices, and historical price snapshots.

2. Platforms, APIs, Databases, and other technologies used (5 points)

List all the platforms, APIs, Databases, and any other technologies you use in your project and where you use them (in what components of your project).

Node.js: Used to run the backend server.

Express.js: Used to create REST API endpoints.

PostgreSQL: Used to persist item metadata, latest market state, and price history.

Skinport API: Used as the external market data source.

Node-cron: Used to schedule automatic refresh jobs.

Dotenv: Used to load environment variables.

Pg: Used to connect the Node.js backend to PostgreSQL.

Cors: Used to allow frontend access to the backend API.

Morgan: Used for HTTP request logging.

3. Execution-based Functional Testing (10 points)

*Describe how/if you performed functional testing for your project (i.e., tested for the **functional requirements** listed in your RD).*

Functional testing was performed by running the backend locally and sending HTTP requests to the API endpoints using curl and browser-based requests. The `/api/v1/items` endpoint was tested with no filters, with search queries, with pagination parameters, and with filters such as weapon, exterior, StatTrak status, minimum price, and maximum price.

The refresh functionality was tested by starting the backend server and verifying that the scheduled job fetched data from Skinport, normalized the items, removed duplicate market hash names, and inserted or updated records in PostgreSQL. Database queries were used to confirm that the `items`, `item_latest`, and `item_price_history` tables were populated correctly.

The health endpoints were tested by requesting `/api/v1/health` and `/api/v1/db` and confirming that the backend returned service and database status information.

Some testing commands used:

```
curl http://localhost:4000/api/v1/health
curl http://localhost:4000/api/v1/db
curl "http://localhost:4000/api/v1/items?limit=10"
curl "http://localhost:4000/api/v1/items?q=ak&limit=10"
curl "http://localhost:4000/api/v1/items?weapon=AK-47&limit=10"
curl "http://localhost:4000/api/v1/items?st=true&limit=10"
curl "http://localhost:4000/api/v1/items?minPrice=10&maxPrice=100&limit=10"
```

4. Execution-based Non-Functional Testing (10 points)

*Describe how/if you performed non-functional testing for your project (i.e., tested for the **non-functional requirements** listed in your RD).*

Non-functional testing focused on performance, reliability, and data integrity. Pagination was tested by requesting limited result sets from the `/api/v1/items` endpoint to ensure the backend did not return the entire dataset at once. Search and filtering were tested against the database to confirm that the API remained usable with a large number of item records.

Reliability was tested by observing scheduled refresh behavior and confirming that overlapping refresh jobs were skipped when a refresh was already in progress. Transaction behavior was also tested by reviewing the refresh process, which begins a database transaction, commits successful updates, and rolls back on failure.

Data integrity was tested by confirming that duplicate logical items were not created for the same `market_hash_name`. The database schema uses unique item metadata and separates stable item information from latest market data and historical snapshots.

5. Non-Execution-based Testing (10 points)

Describe how/if you performed non-execution-based testing (such as code reviews/inspections/walkthroughs).

Non-execution-based testing was performed through code reviews, manual inspections, and team walkthroughs. The team reviewed route logic, database queries, refresh job behavior, and schema design to identify potential issues before merging changes. The code was organized into separate modules for routes, services, database access, scheduled jobs, and application setup, making the system easier to inspect.

The team also reviewed API response formats to ensure the frontend could consume the returned item data. Pull requests and teammate reviews were used to check whether new backend changes matched the project requirements and did not break existing functionality.